

AIMLCZG546 · SESSION 7 · COMPANION READER

Agentic AI and Architectural Patterns

SAGA Orchestration, Choreography, and the Blackboard Pattern

Session 7 · Read alongside the lecture slides · Every pattern explained with worked scenarios

How to use this companion. **Blue** = intuition. **Amber** = everyday analogy. **Green** = worked scenario. **Red** = pitfall. **Purple** = one line to remember.

1 Fundamentals of Agentic AI

Agentic AI refers to AI systems that pursue goals autonomously through multi-step reasoning, planning, tool use, and self-correction — without requiring continuous human direction.

Three complementary definitions:

- **Nvidia:** “Agentic AI uses sophisticated reasoning and iterative planning to autonomously solve complex, multi-step problems.”
- **OpenAI:** “Agentic AI systems are AI systems that can pursue complex goals with limited direct supervision.”
- **Academic:** A system based on a foundation model that conducts complex reasoning including planning and reflection to solve tasks requiring interaction with an environment or elaborate tool use.

The intuition

A chatbot answers questions. An agentic AI *acts* in the world. You tell a chatbot: “What restaurants are near me?” You tell an agent: “Book me a table for two tonight near my office.” The agent looks up restaurants, checks availability, compares reviews, selects one, and books — all without further input from you.

Agentic AI = Generative AI + Reasoning + Tools + Feedback Loops

2 Chatbots vs Agentic AI

Property	Chatbot	Agentic AI
Interaction	Reactive: responds when asked	Proactive: pursues goal until complete
Planning	None	Multi-step planning and sub-goal decomposition
Tool use	None (language only)	Calculators, APIs, databases, code execution
Memory	Stateless (forgets each turn)	Maintains state across many steps
Error recovery	Cannot retry or revise	Detects failure, replans, retries
Scope	Single question	Complex, multi-day tasks

★ Everyday picture

A chatbot is a vending machine: put in a request, get a response, done. An agent is a human assistant: you brief them on a goal, they go figure it out over several days, checking back with you only when they are stuck.

3 LLM as Reasoning Engine

Modern agents use **Large Language Models** as their central reasoning engine. The LLM provides five critical capabilities:

1. **Chain-of-thought reasoning** — the LLM breaks a problem into sequential thinking steps before acting: “First I need X, then I can use X to get Y, then Y will allow me to do Z.”
2. **Instruction following** — converts natural language goals into a sequence of executable actions.
3. **Generalisation** — applies knowledge learned during training to new, unseen tasks without retraining.
4. **Tool use via JSON** — the LLM formats its intentions as structured JSON that external services can parse and execute.
5. **Self-correction** — the LLM evaluates its own outputs, detects errors, and generates revised responses.

3.1 Memory in Agentic Systems

Agents need to remember across multiple steps. Four types of memory:

- **In-context memory** — the current conversation window; everything the LLM can see right now.
- **External memory** — databases, vector stores, and knowledge graphs queried at runtime.
- **Procedural memory** — the tools, APIs, and skills the agent knows how to use.
- **Episodic memory** — logs of past interactions and outcomes used to improve future behaviour.

3.2 Tool Use and JSON Calls

The standard format for an agent tool call:

```
{"tool_name": "function_name",  
  "arguments": {"param1": "value1", "param2": "value2"}}
```

Example: agent needs the weather in Bangalore:

```
{"tool_name": "get_weather", "arguments": {"city": "Bangalore"}}  
→ {"tool_result": "Temperature: 28°C, Condition: Partly Cloudy"}
```

The LLM reads the tool result and incorporates it into its next reasoning step.

4 SAGA Pattern: Distributed Transactions

SAGA solves the following problem: how do you execute a transaction that spans multiple microservices, each with its own database, when traditional two-phase commit (2PC) is too slow and fragile for distributed systems?

The intuition

In traditional (single-database) systems, a transaction is all-or-nothing: either all steps commit, or all roll back automatically. In a distributed system, “automatic rollback” doesn’t exist — each service committed independently. SAGA provides the rollback manually through **compensating transactions**.

A SAGA is a sequence of local transactions. If step k fails, the SAGA executes compensating transactions for steps $k - 1, k - 2, \dots, 1$ to restore the original state.

Worked example: E-commerce order SAGA: orchestration

Order placed: Running Shoes, Qty 1, Price \$150. Four services involved.

Step 1 — Order Service: Creates order in PENDING state.

Step 2 — Orchestrator calls Inventory Service: Reserve 1 unit of “Running Shoes, Size 10”.

Status: **SUCCESS** — inventory reserved.

Step 3 — Orchestrator calls Payment Service: Debit \$150 from customer’s account.
Status: **SUCCESS** — payment deducted.

Step 4 — Orchestrator calls Delivery Service: Schedule delivery for tomorrow.
Status: **FAILURE** — no delivery slots available for this pincode.

Compensation begins (reverse order):

C3 — Payment Refund Service: Refund \$150. Status: **SUCCESS**.

C2 — Inventory Release Service: Unreserve the shoe unit. Status: **SUCCESS**.

C1 — Order Service: Mark order as **CANCELLED**.

Customer experience: Order attempt is declined with a message: “No delivery available to your area. No charge has been applied.” All data is consistent even though four separate databases were touched.

5 SAGA Choreography (Prompt Chaining)

Choreography is the decentralised variant of SAGA. There is no central orchestrator. Each service listens for events and publishes its own events when its local transaction completes. The workflow emerges from the event chain.

In Agentic AI, choreography is called **Prompt Chaining**: a sequence of LLM calls where each call’s output is the next call’s input.

Worked example: Code review agent: prompt chaining

User: “Review my Python code for security vulnerabilities.”

Agent 1 (Parser) receives the code. Publishes event: `code.parsed` with AST.

Agent 2 (Bug Detector) subscribes to `code.parsed`. Analyses AST. Finds: SQL injection on line 47, hardcoded password on line 12, unvalidated input on line 83. Publishes event: `bugs.found` with severity scores.

Agent 3 (Formatter) subscribes to `bugs.found`. Generates a markdown report with

line numbers, severity, and suggested fixes. Publishes: `review.complete`.

Rollback scenario: Agent 2 detects a critical vulnerability but the code context is ambiguous. It publishes `clarification.needed` instead of `bugs.found`. The workflow pauses. Agent 1 re-parses with additional context from the user. Agent 2 retries. This is a compensating transaction in the prompt-chaining context.

✓ Key takeaway

Choreography scales well and is loosely coupled, but debugging is harder (no central log of the full workflow). Orchestration is easier to debug but creates a central bottleneck.

6 Blackboard Pattern

The **Blackboard** pattern solves complex problems that have no predetermined algorithm and require contributions from multiple independent specialists.

The intuition

Imagine a research team solving a mystery. They gather around a physical blackboard. One person writes “Patient has fever.” Another adds “Blood tests show elevated WBC.” A third writes “Patient is allergic to penicillin.” The team leader synthesises: “Bacterial infection — prescribe amoxicillin.” No single person could solve it alone; the solution emerged from collective contributions.

6.1 Three Core Components

1. **The Blackboard** — a shared, central memory repository. Holds: current problem state, partial solutions, hypotheses, intermediate results.
2. **Knowledge Sources (Specialist Agents)** — independent agents with specific expertise. Each monitors the blackboard for updates matching its domain and writes its contribution when it can add value.
3. **Controller / Scheduler** — monitors the blackboard, determines which knowledge source should act next, resolves conflicts, decides when a solution is complete.

Worked example: Medical diagnosis with the Blackboard pattern

Problem posted on blackboard: “Patient: fever 38.5°C, dry cough for 3 days, fatigue.”

Symptom Analyzer (Knowledge Source 1) reads blackboard. Contribution: “Symptom cluster consistent with respiratory tract infection. Confidence: 0.72.” Writes to blackboard.

Controller reads blackboard. Determines Lab Data Agent should act next (blood tests pending).

Lab Data Agent (Knowledge Source 2) reads blood test results from hospital system. Contribution: “WBC = 14,000/μL (elevated, normal: 4,500–11,000). Neutrophil ratio: 82 % (bacterial indicator).” Writes to blackboard.

Drug Interaction Checker (Knowledge Source 3) reads patient medication list. Contribution: “Patient is on warfarin (blood thinner). NSAIDs (ibuprofen, aspirin) contraindicated. Penicillin safe.” Writes to blackboard.

Controller detects sufficient evidence. Calls Diagnosis Coordinator.

Diagnosis Coordinator (Knowledge Source 4) synthesises all contributions. Final diagnosis written to blackboard: “Community-acquired pneumonia (bacterial). Prescription: Amoxicillin 500mg three times daily for 7 days. Avoid NSAIDs. Follow up in 5 days.”

In Agentic AI: The LLM acts as Controller/Scheduler. Specialist agents are Knowledge Sources. The conversation context window or a shared vector store is the Blackboard.

⚠ Watch out

Blackboard systems can loop indefinitely if no agent can make progress and the controller does not recognise this condition. Always define a termination criterion: either “solution found” or “maximum iterations reached, escalate to human.”

✓ Key takeaway

The Blackboard pattern is ideal for open-ended problems where the solution must emerge from the combined expertise of multiple specialists — exactly the shape of complex agentic AI tasks.